



SwalaSMS Developer API

Integration Knowledge Base

Prepared for FIXO — Home Services Made Simple

API version v1.0.0 · 3 groups · 9 endpoints

Base URL <https://swalasms.com/api/v1>

Source: <https://swalasms.com/sms/developer/docs>

Compiled 14 September 2026 · Document revision 1.0

Contents

How to use this document.....	4
1. Platform overview.....	5
1.1 What the API covers.....	5
1.2 The five gates every send passes.....	5
1.3 Sandbox and live modes.....	5
2. Authentication and API keys.....	7
2.1 Base URL.....	7
2.2 Bearer token.....	7
2.3 Key prefixes.....	7
2.4 Scopes.....	7
2.5 Address restrictions.....	8
2.6 Key handling rules.....	8
3. Request and response conventions.....	9
3.1 Every response is JSON.....	9
3.2 Phone number format.....	9
3.3 Why sends return 202.....	10
3.4 Message statuses.....	10
3.5 Idempotency.....	10
3.6 Rate limiting.....	11
3.7 Malformed JSON.....	11
4. Messaging endpoints.....	12
4.1 Check SMS balance.....	12
4.2 Send a single SMS.....	13
4.3 List messages and their delivery status.....	16
4.4 Send bulk SMS.....	18
4.5 Check one message.....	20
5. Sender ID endpoints.....	22
5.1 Get sender ID requirements.....	22
5.2 List sender IDs.....	25
5.3 Request a sender ID.....	27
6. Account endpoint.....	30
6.1 Read the account profile.....	30
7. Webhooks.....	32
7.1 Setting one up.....	32
7.2 Events.....	32
7.3 Payload.....	32
7.4 Verifying the signature.....	32
7.5 Acknowledgement and retries.....	33
8. Error model.....	34
8.1 Status codes.....	34
8.2 Which failures are worth retrying.....	34
8.3 Two kinds of 422.....	35
9. Playbooks.....	36
9.1 First integration, from nothing to a live message.....	36

9.2 Getting a sender ID approved.....	36
9.3 Running a bulk campaign.....	37
9.4 Reconciling delivery.....	37
10. Troubleshooting.....	38
11. Quick reference.....	39
11.1 All endpoints.....	39
11.2 Alternate paths.....	39
11.3 Limits.....	39
12. Glossary.....	41
Appendix A — Document control.....	42
Maintenance notes.....	42

How to use this document

This knowledge base is a consolidated, offline-readable reference for the SwalaSMS Developer API v1.0.0. It restates the live documentation in a structure suited to onboarding, internal support and integration review, and adds implementation guidance drawn from the behaviour the API describes.

It is organised into four layers:

- 1. Foundations** — base URL, authentication, keys and scopes, request and response conventions. Read this once before writing any code.
- 2. Endpoint reference** — all nine endpoints grouped as Messaging, Sender IDs and Account, each with its scope, parameters, a worked example, the success response and every documented failure.
- 3. Delivery and failure handling** — webhooks, signature verification, retry behaviour, and the shared error model.
- 4. Playbooks and checklists** — ordered procedures for go-live, sender ID approval, bulk campaigns and troubleshooting.

Authoritative source. Where this document and the live reference at <https://swalasms.com/sms/developer/docs> disagree, the live reference wins. An OpenAPI description is published at <https://swalasms.com/sms/developer/openapi.json> and is the best input for generating client SDKs.

Never paste a real key into this file. Every key in these pages is a placeholder. A key is displayed once, at creation. Store live keys in a secrets manager or environment variables, never in documents, tickets, chat messages or source control.

1. Platform overview

The SwalaSMS Developer API lets an external system send and track SMS messages through the same routing, sender ID, wallet, spam-control and country-credit rules that the SwalaSMS web portal applies. Nothing is relaxed because a request arrived over the API rather than through the dashboard: a message that the portal would refuse is refused identically over HTTP.

That symmetry is the single most useful fact about the platform. It means portal behaviour is a reliable guide to API behaviour, and it means an integration cannot bypass compliance controls by talking to the API directly.

1.1 What the API covers

Group	Endpoints	Purpose
Messaging	5	Send single and bulk SMS, check delivery status, read the credit balance.
Sender IDs	3	Read the requirements, submit a sender ID for review, track approval.
Account	1	Read the account profile, verified routes and per-country balances.

1.2 The five gates every send passes

Before a message is accepted, SwalaSMS checks all of the following. A failure in any one produces a `422` whose `message` names the cause.

- 5. Active account** — the API key owner's account must be in good standing.
- 6. Approved sender ID** — the `sender_id` must be a name this account owns and that has been approved for the destination country.
- 7. Verified SMS route** — a route must be verified for the destination country.
- 8. Customer SMS credit** — the country wallet must hold enough SMS units.
- 9. Provider route and provider credit** — an upstream route must be available with credit on it.

A sixth check applies to content rather than configuration: the message body is evaluated by the **spam policy** before it is accepted, and recipient opt-out is honoured.

Checked inside a transaction. The wallet and provider-credit checks run inside a database transaction at send time. `GET /balance` is useful for warning a user before a campaign is queued, but it is an advisory read, not a reservation — the authoritative check happens when the send is processed.

1.3 Sandbox and live modes

Every API key is either sandbox or live, and the mode is visible in the key prefix and echoed in send responses as `mode`.

	Sandbox <code>swl_test</code>	Live <code>swl_live</code>
Real SMS sent	No	Yes
SMS credits consumed	No	Yes
Response shape	Realistic queued / delivered responses	Real queued responses
Status lookups	Answers <code>delivered</code> immediately without looking anything up	Reflects actual carrier state
When to use	All development and automated tests	Only once sender IDs, route, wallet and provider credits are ready

Build end to end before sending anything real. Because a sandbox key answers `delivered` immediately, a full integration — send, poll, webhook handling, reconciliation — can be exercised before a single real message leaves the platform.

2. Authentication and API keys

2.1 Base URL

```
https://swalasms.com/api/v1
```

Every path in this document is relative to that base unless written out in full.

2.2 Bearer token

Every request must carry the key issued with your application as a bearer token in the `Authorization` header.

```
Authorization: Bearer swl_live_YOUR_API_KEY
```

There is no alternative scheme — no query-string key, no basic auth. A request without this header is answered with `401`.

2.3 Key prefixes

Prefix	Meaning
<code>swl_live_</code>	Live key. Sends real SMS and consumes real credits.
<code>swl_test_</code>	Sandbox key. Realistic responses, no real SMS, no credits consumed.
<code>YOUR_API_KEY</code>	The documentation placeholder. It is not a key and is refused.

A key is shown once. The full key value is displayed only at the moment it is created. If it is lost it cannot be recovered — a new key must be issued and the old one revoked. Manage keys at <https://swalasms.com/sms/api-keys>.

2.4 Scopes

A key carries only the permissions granted to its application. Each endpoint names the scope it needs; a key without that scope is answered with `401` — the same response as a key that is missing, expired or revoked. The API deliberately does not distinguish between these cases in the response.

Scope	Grants access to
<code>sms:read</code>	Check SMS balance · List messages · Check one message
<code>sms:send</code>	Send a single SMS · Send bulk SMS
<code>sender-ids:read</code>	Get sender ID requirements · List sender IDs
<code>sender-ids:write</code>	Request a sender ID
<code>profile:read</code>	Read the account profile

Grant the narrowest set a component actually needs. A reporting dashboard that only displays delivery statistics should hold `sms:read` and nothing else; a key that can read reports but cannot spend credit is a materially safer key to deploy.

2.5 Address restrictions

A key may be restricted to a set of calling addresses. A request from an address the key is not allowed from is refused with `401`, identically to an unauthorised key. If a key that worked in development fails in production with `401` and the scope is correct, an address restriction is the first thing to check.

2.6 Key handling rules

- Store keys in environment variables or a secrets manager — never in source control, documents, tickets or chat.
- Issue separate keys per application and per environment, so one can be revoked without disrupting the others.
- Revoke immediately on suspected exposure; a revoked key starts failing with `401` at once.
- Never send a key from browser or mobile client code. All SwalaSMS calls belong on a server you control.
- Log the `request_uid` from responses, never the key itself.

3. Request and response conventions

3.1 Every response is JSON

Success and failure share a predictable envelope.

```
// success
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "data": { ... }
}

// failure
{
  "success": false,
  "request_uid": "7b19fc2a-6d2b-47c7-95fc-d49ae6d03528",
  "message": "Unauthorized API key."
}
```

Field	Always present	Notes
<code>success</code>	Yes	Boolean. Branch on this, not on the HTTP status alone.
<code>request_uid</code>	Yes	Identifier for this attempt. Log it on every call; quote it when contacting support.
<code>data</code>	On success	Object or array, depending on the endpoint.
<code>message</code>	On failure	Human-readable explanation. Safe to surface to an operator.
<code>errors</code>	On 422 validation	Field-keyed map of validation messages.
<code>error</code>	On some failures	Machine-readable code, e.g. <code>idempotency_key_reused</code> .
<code>mode</code>	On sends	live or sandbox .

Log the request_uid. It is the one value that lets SwalaSMS support trace a specific attempt through their systems. An integration that discards it turns every support conversation into guesswork.

3.2 Phone number format

Send recipients in E.164 format: a leading plus, the country calling code, then the subscriber number with no spaces, dashes or brackets.

Correct	Incorrect
<code>+255712345678</code>	<code>0712345678</code> — national format, no country code
<code>+255755000001</code>	<code>255 712 345 678</code> — spaces

Correct	Incorrect
	+255-712-345-678 — separators

Normalise at your edge. Users type numbers in national format. Convert to E.164 at the point of capture, store the E.164 form, and send that. Note that webhook payloads may carry `to` in national form (`0712345678`), so match callbacks on `message_id` rather than on the number.

3.3 Why sends return 202

A successful send returns `202 Accepted`, not `200 OK`. The distinction is meaningful: `202` says the request was accepted and queued, not that the SMS was delivered. Delivery is asynchronous and the carrier decides the outcome seconds or minutes later.

An integration therefore learns the outcome in one of two ways:

- **Webhooks** — SwalaSMS posts to your endpoint when a message reaches a final state. This is the recommended path.
- **Polling** — read `GET /sms/messages/{uid}` for one message, or `GET /sms/messages` for a filtered list.

3.4 Message statuses

Status	Final	Meaning
<code>queued</code>	No	Accepted by SwalaSMS and waiting to be dispatched.
<code>processing</code>	No	Being handed to a provider.
<code>sent</code>	No	Handed to the carrier; delivery not yet confirmed.
<code>delivered</code>	Yes	The carrier confirmed delivery. <code>delivered_at</code> is set at that moment.
<code>failed</code>	Yes	Ended in failure. <code>failure_reason</code> explains why, written for a person.
<code>scheduled</code>	No	Held for a future <code>scheduled_at</code> time.

The normal progression is `queued` → `sent` → `delivered`, or it ends at `failed`. Filtering on the list endpoint accepts `queued`, `sent`, `delivered` and `failed`.

3.5 Idempotency

Both send endpoints accept an `Idempotency-Key` header. It exists so that a retry — after a timeout, a dropped connection or a crashed worker — cannot send the same SMS twice.

The three outcomes

Situation	Result
Key never used before	Message is queued normally.
Same key, identical content	The first message is returned with <code>idempotent_replay: true</code> . Nothing is sent again.
Same key, different content	<code>409</code> with <code>error: idempotency_key_reused</code> . Nothing is sent. The

Situation	Result
	response names the earlier message UID and its creation time.

- Keys are remembered for **24 hours**.
- Generate a fresh key per message or per campaign — a UUID is the obvious choice.
- Derive it from something stable in your own system (an order ID, a notification row ID) so a retry naturally reproduces the same key.
- Omitting the header entirely is permitted, but then a retry may genuinely send twice.

One key, one message. A recycled key is not a shortcut to resending. Reusing a key for different content is refused outright rather than answered with the earlier message — the API treats it as a programming error, because it almost always is.

3.6 Rate limiting

The limit is applied **per key, per minute**. Exceeding it returns [429](#) with the message [API rate limit exceeded](#). Implement exponential backoff with jitter, and treat [429](#) as retryable — unlike [422](#), which will fail identically on every attempt.

Repeating the same body and sender through the single-message endpoint is specifically rate-limited. For one body to many recipients, use [POST /sms/send-bulk](#), which is designed for it.

3.7 Malformed JSON

A body that claims to be JSON but cannot be parsed is answered with [400](#) and says so, rather than reporting every field as missing. The reasoning is that if the payload cannot be parsed, none of its fields exist as far as the server is concerned.

The usual cause is curly quotes. Pasting a payload out of a document or a chat window commonly replaces straight quotes with typographic ones. If a [400](#) appears after copy-and-paste, check the quotes first, then unterminated strings and missing commas.

4. Messaging endpoints

Five endpoints cover sending and tracking. Two spend credit (`sms:send`); three are read-only (`sms:read`).

4.1 Check SMS balance

GET `/api/v1/balance` scope: `sms:read`

Also answers at `/api/v1/sms/balance` with identical behaviour.

Returns the customer SMS credit balance for the API key owner. The top-level `balance` is the total available SMS units across all enabled country wallets. The `countries` array breaks that down per country with the wallet UID, ISO country code, calling code and remaining units.

Request

```
curl -X GET "https://swalasms.com/api/v1/balance" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY"
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "data": {
    "balance": "value",
    "currency": "SMS",
    "countries": [
      {
        "wallet_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
        "country": "TZ",
        "calling_code": "+255",
        "balance": "value"
      }
    ]
  }
}
```

Failures

Status	Cause
401	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.
429	Too many requests for this key in one minute.

Advisory, not a reservation. Use this to warn a user before a campaign is queued. The send endpoints still perform the final wallet and provider-credit checks inside a database transaction, so a balance read does not guarantee a subsequent send will succeed.

4.2 Send a single SMS

POST /api/v1/sms/messages scope: **sms:send**

Also answers at </api/v1/sms/quick-message> with identical behaviour.

Queues one SMS for delivery through the active SwalaSMS routing engine, under the same rules a message gets when it is composed in the portal.

Prerequisites

The API key owner must have an active account, an approved sender ID assigned to the account, a verified SMS route for the destination country, sufficient customer SMS credit, and an available provider route with provider credit. The message body is checked by the spam policy before it is accepted.

Body

Field	Required	Type	Constraint
<code>recipient</code>	Yes	string	Max 32 characters. One phone number in E.164 format.
<code>sender_id</code>	Yes	string	Max 32 characters. One of your own approved sender ID names.
<code>body</code>	Yes	string	Max 1600 characters.

Headers

Header	Required	Notes
<code>Authorization</code>	Yes	<code>Bearer swl_live_...</code>
<code>Content-Type</code>	Yes	<code>application/json</code>
<code>Idempotency-Key</code>	Optional	Must be unique per message. Remembered for 24 hours.

Request

```
curl -X POST "https://swalasms.com/api/v1/sms/messages" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY" \
-H "Content-Type: application/json" \
-H "Idempotency-Key: 35857c95-a16b-4236-8f3b-7c611b620f97" \
-d '{
  "recipient": "+255712345678",
  "sender_id": "YOURBRAND",
  "body": "Your order is ready for pickup."
}'
```

Success response — 202 Accepted

```
{
  "success": true,
  "request_uid": "7b19fc2a-6d2b-47c7-95fc-d49ae6d03528",
  "mode": "live",
  "data": {
    "uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
    "recipient": "+255712345678",
```

```

    "sender_id": "SWALASMS",
    "units": 1,
    "channel": "api",
    "status": "queued",
    "queued_at": "2026-08-25T09:00:00+00:00",
    "sent_at": null,
    "delivered_at": null,
    "failure_reason": null
  }
}

```

Persist `data.uid` against your own record. It is the handle for every later status lookup and the value webhooks carry as `message_id`.

Failures

Status	Cause
400	The body is not valid JSON, so no field was read and nothing was sent.
401	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.
409	The <code>Idempotency-Key</code> was already used for a different request. Response carries <code>error: idempotency_key_reused</code> , <code>earlier_uid</code> and <code>earlier_created_at</code> .
422 (validation)	A field is missing or malformed. Response carries a field-keyed <code>errors</code> map.
422 (rule)	Understood but refused: unapproved sender ID, no verified route, no credit, or the spam policy.
429	Too many requests for this key in one minute.

Example — 422 validation

```

{
  "success": false,
  "request_uid": "7b19fc2a-6d2b-47c7-95fc-d49ae6d03528",
  "message": "The recipient field is required. (and 1 more error)",
  "errors": {
    "recipient": [
      "The recipient field is required. Send one phone number in E.164 format, for example +255712345678."
    ],
    "sender_id": [
      "The sender_id field is required. Use an approved sender ID name from GET /api/v1/sender-ids."
    ]
  }
}

```

Example — 409 idempotency key reused

```

{
  "success": false,
  "request_uid": "7b19fc2a-6d2b-47c7-95fc-d49ae6d03528",
  "error": "idempotency_key_reused",
  "message": "The Idempotency-Key \"3f1c...\" was already used for a different message on 2026-08-25T09:00:00+00:00. ...",
}

```

```
"data": {
  "idempotency_key": "3f1c...",
  "earlier_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "earlier_created_at": "2026-08-25T09:00:00+00:00"
}
```

Do not loop this endpoint for a broadcast. For one message to many recipients use [POST /api/v1/sms/send-bulk](#). Repeating the same body and sender through the single-message endpoint is rate-limited.

4.3 List messages and their delivery status

GET /api/v1/sms/messages scope: **sms:read**

Every message this account has sent, newest first, with what became of it. Because a send returns **queued** and the carrier decides the outcome later, this is how an integration finds out.

Query parameters

Name	Type	Notes
status	string	Delivery status: queued , sent , delivered or failed .
channel	string	Where the message came from: api or web . Useful for separating your integration's traffic from messages staff sent in the portal.
from	string	Only messages created on or after this date (YYYY-MM-DD), inclusive.
to	string	Only messages created on or before this date (YYYY-MM-DD), inclusive.
per_page	integer	Messages per page. Defaults to 50, capped at 100.
page	integer	Page number, starting at 1.

Request

```
curl -X GET "https://swalasms.com/api/v1/sms/messages?status=delivered\
&channel=api&from=2026-08-01&to=2026-08-31&per_page=50&page=1" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY"
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "data": [
    {
      "uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
      "recipient": "+255712345678",
      "sender_id": "SWALASMS",
      "units": 1,
      "channel": "api",
      "status": "delivered",
      "queued_at": "2026-08-25T09:00:00+00:00",
      "sent_at": "2026-08-25T09:00:02+00:00",
      "delivered_at": "2026-08-25T09:00:07+00:00",
      "failure_reason": null
    }
  ],
  "meta": {
    "current_page": 1,
    "per_page": 50,
    "total": 42,
    "last_page": 3
  }
}
```


Each entry carries `status`, `queued_at`, `sent_at`, `delivered_at`, `failure_reason` and the billed `units`. Page through using `meta.last_page`.

Failures

Status	Cause
401	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.
429	Too many requests for this key in one minute.

4.4 Send bulk SMS

POST /api/v1/sms/send-bulk scope: **sms:send**

Queues the same SMS body to many recipients in one API call. Use it when your application already has the recipient list and each number is ready in E.164 format. The request accepts **up to 1000 recipients**.

SwalaSMS still creates one local message per recipient, so balance accounting, reports, idempotency and delivery webhooks keep working exactly as they do in the portal.

Body

Field	Required	Type	Constraint
<code>recipients</code>	Yes	array	1 to 1000 recipient phone numbers. May also be sent as one comma-separated string.
<code>message</code>	Yes	string	The same SMS body sent to every recipient. Max 1600 characters.
<code>sender_id</code>	Yes	string	Your approved SwalaSMS sender ID name. Max 32 characters.
<code>campaign_name</code>	Optional	string	A name for this campaign in reports. Max 120 characters.
<code>scheduled_at</code>	Optional	string	Future ISO-8601 date/time, or omit / <code>null</code> to send now.

Field aliases. `body` is accepted in place of `message`, and `name` in place of `campaign_name`, so an integration written against the single-send endpoint keeps working when it is pointed at bulk.

Request

```
curl -X POST "https://swalasms.com/api/v1/sms/send-bulk" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY" \
-H "Content-Type: application/json" \
-H "Idempotency-Key: d951d535-aca3-427f-b053-d9da1f72dbf3" \
-d '{
  "recipients": [
    "+255755000001",
    "+255755000002",
    "+255755000003"
  ],
  "message": "Flash sale! 30% off today only.",
  "sender_id": "YOURBRAND",
  "campaign_name": "June Flash Sale",
  "scheduled_at": null
}'
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "mode": "sandbox",
}
```

```

    "data": {
      "uid": "4c3b2a19-5566-7788-99aa-bbccddeeff00",
      "name": "August Promo",
      "status": "queued",
      "recipient_count": 3,
      "queued_count": 3,
      "failed_count": 0,
      "scheduled_at": null,
      "failures": [],
      "results": [
        {
          "recipient": "+255755000001",
          "status": "queued",
          "message_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e"
        }
      ]
    }
  }
}

```

Campaign success is not per-recipient success. The response carries campaign-level counts. An individual recipient can still be skipped if its route, wallet, sender ID, provider credit or spam checks fail — always read `failed_count` and `failures`, not just `success`.

Idempotency for campaigns

The `Idempotency-Key` must be unique **per campaign**, on the same terms as the single send. An identical repeat replays the first campaign with `idempotent_replay: true` and sends nothing; a different campaign under a used key is refused with `409`. Keys are remembered for 24 hours.

Failures

Status	Cause
<code>400</code>	The body is not valid JSON. Reported as such rather than as a list of missing fields.
<code>401</code>	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.
<code>409</code>	<code>idempotency_key_reused</code> — the key was already used for a different campaign.
<code>422</code> (validation)	A field is missing or malformed, e.g. <code>message</code> absent or <code>recipients</code> not an array.
<code>422</code> (rule)	Unapproved sender ID, no verified route, no credit, or the spam policy.
<code>429</code>	Too many requests for this key in one minute.

4.5 Check one message

GET /api/v1/sms/messages/{uid} scope: **sms:read**

The delivery status of a single message, by the **uid** the send returned. **status** moves **queued** → **sent** → **delivered**, or ends at **failed** with a **failure_reason** written for a person. **delivered_at** is set the moment the carrier confirms delivery.

Path parameter

Name	In	Type	Notes
uid	path	string	The message UID returned by a send. Must be a valid UUID.

Request

```
curl -X GET "https://swalasms.com/api/v1/sms/messages/{uid}" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY"
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "data": {
    "uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
    "recipient": "+255712345678",
    "sender_id": "SWALASMS",
    "units": 1,
    "channel": "api",
    "status": "delivered",
    "queued_at": "2026-08-25T09:00:00+00:00",
    "sent_at": "2026-08-25T09:00:02+00:00",
    "delivered_at": "2026-08-25T09:00:07+00:00",
    "failure_reason": null,
    "campaign_uid": "4c3b2a19-5566-7788-99aa-bbccddeeff00",
    "campaign_name": "August Promo",
    "failure_context": null,
    "delivery_report": {
      "checked_at": "2026-08-25T09:05:00+00:00",
      "status": "delivered",
      "delivery_status": "DELIVRD"
    }
  }
}
```

This endpoint returns more than the list view: **campaign_uid**, **campaign_name**, **failure_context** and the raw **delivery_report** from the carrier. Which provider carried the message is deliberately not exposed.

Failures

Status	Cause
401	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.

Status	Cause
404	No message with that UID belongs to this account.
422	The UID is not a valid UUID.
429	Too many requests for this key in one minute.

404 and 422 are told apart deliberately. An invalid UID is answered with a validation error; a valid UID belonging to another account is answered with **404**. The distinction means a wrong ID reads as a mistake to fix, never as a hint about somebody else's traffic.

Sandbox shortcut. A sandbox key answers **delivered** immediately without looking anything up, so an integration can be built end to end before a real message is ever sent.

5. Sender ID endpoints

A sender ID is the name a recipient sees as the origin of the message. It is registered **per country**, approved manually, and cannot be used for sending until it is approved. These three endpoints mirror the web flow exactly.

Always start with requirements. Call `GET /sender-ids/requirements` before submitting anything. Every refusal the submit endpoint can produce is something that call would have told you in advance — eligibility, permitted countries, mandatory documents and naming rules all differ per platform and per account.

5.1 Get sender ID requirements

GET `/api/v1/sender-ids/requirements` scope: **sender-ids:read**

Also answers at `/api/v1/sms/sender-ids/requirements` with identical behaviour.

What the response tells you

Key	Meaning
<code>eligibility.can_request</code>	<code>false</code> means the account has not bought enough SMS credits yet. <code>required_units</code> and <code>minimum_purchase_amount</code> say how much is needed. Do not attempt the request.
<code>countries[]</code>	The only countries this account may request for. A country appears here once a phone number has been verified for it under SMS Routes.
<code>documents[]</code>	Every entry with <code>is_required: true</code> must be attached when you request. Use its <code>uid</code> as the <code>requirement_uid</code> .
<code>documents_required</code>	Boolean shortcut. <code>false</code> on most platforms — a request with no documents is then complete, not incomplete.
<code>name_reservation</code>	Whether the optional name claim is supported, the field names to use, accepted formats and the size ceiling.
<code>purposes</code>	The allowed purpose values. Send exactly one as <code>purpose</code> .
<code>sender_id_rules</code>	3 to 11 characters, letters and numbers with single spaces between words, plus a list of reserved words that are always refused.
<code>sample_message_rules</code>	50 to 500 characters. Reviewers read it.

Request

```
curl -X GET "https://swalasms.com/api/v1/sender-ids/requirements" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY"
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "data": {
```

```

    "eligibility": {
      "can_request": true,
      "purchased_units": 5000,
      "required_units": 50,
      "currency": "TZS",
      "minimum_purchase_amount": "125000.00",
      "price_per_unit": "25.00",
      "reason": null
    },
    "countries": [
      {
        "uid": "ab98170f-3c7a-4cbe-8e3f-71a424b127e6",
        "name": "Tanzania",
        "iso2": "TZ",
        "calling_code": "+255"
      }
    ],
    "documents": [],
    "documents_required": false,
    "name_reservation": {
      "supported": true,
      "required": false,
      "request_field": "protection_requested",
      "documents_field": "protection_documents_base64",
      "max_documents": 3,
      "accepted_formats": ["pdf", "jpg", "jpeg", "png"],
      "max_size_kb": 2048,
      "description": "Optional. Set protection_requested to true and",
      "...": "attach proof that you own the name to have it reserved."
    },
    "purposes": [
      "Notification", "Transactional", "OTP / Verification",
      "Promotional", "Marketing", "Alerts & Reminders", "Other"
    ],
    "sender_id_rules": {
      "min_length": 3,
      "max_length": 11,
      "pattern": "^[A-Za-z0-9]+( [A-Za-z0-9]+)*$",
      "description": "Letters, numbers, and single spaces between words.",
      "reserved_names": ["SWALASMS", "SWALA"]
    },
    "sample_message_rules": {
      "min_length": 50,
      "max_length": 500,
      "description": "A real example of the messages you will send."
    }
  }
}

```

Naming rules at a glance

Rule	Value
Length	3 to 11 characters
Allowed characters	Letters and numbers, with single spaces between words
Pattern	<code>^[A-Za-z0-9]+([A-Za-z0-9]+)*\$</code>
Reserved names	SWALASMS, SWALA — always refused

Rule	Value
Case	Yours to choose, and carried through to the handset

Failures

Status	Cause
401	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.
429	Too many requests for this key in one minute.

5.2 List sender IDs

GET /api/v1/sender-ids scope: **sender-ids:read**

Also answers at /api/v1/sms/sender-ids with identical behaviour.

Every sender ID this account can send with, plus every request it has made, at any status. This is the endpoint to poll after submitting a request.

Statuses

Status	Meaning
pending	Submitted and awaiting manual review.
approved	Usable. <code>is_usable</code> is <code>true</code> .
rejected	Turned down. <code>rejection_reason</code> explains why.
suspended	Previously approved, now withdrawn from use.

Gate sending on `is_usable`. A sender ID may only be used for sending once `is_usable` is `true`, which corresponds to `status: approved`. Check that flag rather than inferring usability from the presence of a record.

Request

```
curl -X GET "https://swalasms.com/api/v1/sender-ids" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY"
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "data": [
    {
      "uid": "a1b2c3d4-1111-2222-3333-444455556666",
      "sender_id": "SWALASMS",
      "country_uid": "ab98170f-3c7a-4cbe-8e3f-71a424b127e6",
      "country_code": "+255",
      "country_name": "Tanzania",
      "status": "approved",
      "is_default": true,
      "is_global": false,
      "sample_message": "Your order has shipped and is on its way.",
      "is_usable": true,
      "submitted_document_uids": [
        "c3d4e5f6-3333-4444-5555-666677778888"
      ],
      "requested_at": "2026-08-01T10:00:00+00:00",
      "updated_at": "2026-08-02T08:00:00+00:00"
    }
  ]
}
```

Failures

Status	Cause
401	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.
429	Too many requests for this key in one minute.

5.3 Request a sender ID

POST `/api/v1/sender-ids` scope: `sender-ids:write`

Also answers at `/api/v1/sms/sender-ids` with identical behaviour.

Submits a sender ID for manual review, applying exactly the same gates as the web form. Anything refused here would have been refused there.

Body

Field	Required	Notes
<code>sender_id</code>	Yes	The name recipients will see. 3 to 11 characters, letters and numbers, single spaces between words.
<code>sample_message</code>	Yes	A real message you will send from this sender ID. 50 to 500 characters.
<code>purpose</code>	Yes	Exactly one of: Notification, Transactional, OTP / Verification, Promotional, Marketing, Alerts & Reminders, Other.
<code>country_uid</code>	Optional	From GET <code>/sender-ids/requirements</code> . Omit it and the request is registered for Tanzania (<code>ab98170f-3c7a-4cbe-8e3f-71a424b127e6</code>), the only market currently open to sender ID requests.
<code>documents_base64</code>	Optional	Only when a requirement is returned with <code>is_required: true</code> . Entries are <code>{requirement_uid, filename, content_base64}</code> .
<code>protection_requested</code>	Optional	Boolean. <code>true</code> to also claim the name so nobody else may register it.
<code>protection_documents_base64</code>	Optional	Required only when <code>protection_requested</code> is <code>true</code> . Entries are <code>{filename, content_base64}</code> .

Attaching documents

Usually there are none. `documents_required` is `false` for most platforms, and a request with no documents is then a complete request, not an incomplete one. Send documents only for requirements returned with `is_required: true`, in either of two ways:

- **JSON** — `documents_base64: [{ "requirement_uid": "...", "filename": "certificate.pdf", "content_base64": "..." }]`
- **Multipart** — a file part named `documents[<requirement_uid>]`

Accepted formats are `pdf`, `jpg`, `jpeg` and `png`. The size ceiling is given by `max_size_kb` on the requirements response, because it depends on what the server accepts.

Claiming the name

The web form offers a tick box that reserves the name so nobody else may register it. The API takes the same claim as `protection_requested: true` plus at least one supporting document in `protection_documents_base64`, or as multipart `protection_documents[]`.

The claim is optional; the sender ID is granted either way. The claim only decides whether the name is reserved. Ticking it with nothing attached is refused, exactly as on the web.

Request — no documents, the usual case

```
curl -X POST "https://swalasms.com/api/v1/sender-ids" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "sender_id": "YOURBRAND",
  "country_uid": "ab98170f-3c7a-4cbe-8e3f-71a424b127e6",
  "sample_message": "Your order has been confirmed and is being prepared.
    We will send you another message as soon as it is ready for pickup.",
  "purpose": "Notification"
}'
```

Request — with documents and a name claim

```
curl -X POST "https://swalasms.com/api/v1/sender-ids" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "sender_id": "YOURBRAND",
  "country_uid": "ab98170f-3c7a-4cbe-8e3f-71a424b127e6",
  "sample_message": "Your order has been confirmed and is being prepared.",
  "purpose": "Notification",
  "documents_base64": [
    {
      "requirement_uid": "c3d4e5f6-3333-4444-5555-666677778888",
      "filename": "certificate.pdf",
      "content_base64": "JVBERi0xLjQKJelJz9MKMy..."
    }
  ],
  "protection_requested": true,
  "protection_documents_base64": [
    {
      "filename": "brela-certificate.pdf",
      "content_base64": "JVBERi0xLjQKJelJz9MKMy..."
    }
  ]
}'
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "message": "Sender ID request submitted for review.",
  "data": {
    "uid": "a1b2c3d4-1111-2222-3333-444455556666",
    "sender_id": "SWALASMS",
    "country_uid": "ab98170f-3c7a-4cbe-8e3f-71a424b127e6",
    "country_code": "+255",
    "country_name": "Tanzania",
    "status": "approved",
    "is_default": true,
    "is_global": false,
    "sample_message": "Your order has shipped and is on its way.",
  }
}
```

```

    "is_usable": true,
    "submitted_document_uids": [...],
    "requested_at": "2026-08-01T10:00:00+00:00",
    "updated_at": "2026-08-02T08:00:00+00:00"
  }
}

```

Named errors

Status	error	Meaning
422	sender_id_not_eligible	Not enough SMS credits purchased yet.
422	route_not_verified	No verified SMS route for that country.
422	country_not_available	That country is not open on this platform.
409	sender_id_already_requested	This name already exists for this country. Poll GET /sender-ids instead — retrying never succeeds.

Other failures follow the shared model: [400](#) for unparseable JSON, [401](#) for key problems, [422](#) for missing or malformed fields, and [429](#) for rate limiting.

Approval is manual. There is no automatic approval and no callback. Poll [GET /sender-ids](#) until `is_usable` is `true`. Build the waiting period into your go-live plan rather than treating it as an error state.

6. Account endpoint

6.1 Read the account profile

GET /api/v1/sms/profile scope: **profile:read**

Returns who the key belongs to, its verified SMS routes, and the credit balance held for each country. It is the natural first call for a health check or a diagnostic screen: it confirms in a single request that the key is valid, which routes are verified and what is left to spend.

Request

```
curl -X GET "https://swalasms.com/api/v1/sms/profile" \
-H "Authorization: Bearer swl_live_YOUR_API_KEY"
```

Success response

```
{
  "success": true,
  "request_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
  "data": {
    "uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
    "name": "Jane Doe",
    "email": "jane@example.com",
    "phone": "+255712345678",
    "phone_verified_at": "2026-08-25T09:00:00+00:00",
    "country": {
      "uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
      "iso2": "TZ",
      "calling_code": "+255",
      "name": "Jane Doe"
    },
    "balances": [
      {
        "wallet_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
        "country_uid": "9f2c1e3a-4b7d-4e2b-9c3e-2f6a7d8b1c4e",
        "country": "TZ",
        "calling_code": "+255",
        "balance": 500,
        "currency": "SMS"
      }
    ],
    "routes": "value"
  }
}
```

Failures

Status	Cause
401	Key missing, expired, revoked, calling from a disallowed address, or without the required scope.
429	Too many requests for this key in one minute.

Contains personal data. The profile carries the account holder's name, email and phone number. Treat the response as personal data: do not log it wholesale, and do not expose it to end users of your own application.

7. Webhooks

Webhooks push delivery reports to your application as soon as a message reaches a final state. They are the preferred alternative to polling: an integration that relies on webhooks learns outcomes within seconds and makes no wasted requests.

7.1 Setting one up

10. Create an API key first, at <https://swalasms.com/sms/api-keys>.
11. Open SMS Webhooks at <https://swalasms.com/sms/webhooks> and attach one or more endpoints to that key.
12. Copy the signing secret. It is issued once, when the endpoint is created or regenerated. Store it securely.

Webhooks are scoped to a key. Only messages sent with a given API key are posted to that key's webhooks. If you run separate keys per environment or per service, each needs its own endpoint configured.

7.2 Events

Event	When it is sent
<code>sms.delivered</code>	The message was delivered.
<code>sms.failed</code>	The message reached a final failed state.

Only final states are pushed. Intermediate transitions such as `queued` → `sent` do not generate a callback; read those from the status endpoints if you need them.

7.3 Payload

```
{
  "event": "sms.delivered",
  "message_id": "9757991f-0000-4000-9000-000000000000",
  "to": "0712345678",
  "status": "delivered",
  "sender_id": "MYBRAND",
  "error": null,
  "delivered_at": "2026-08-30T09:15:00+03:00",
  "timestamp": 1788069700
}
```

`message_id` corresponds to the `uid` the send returned — match on it. Note that `to` may arrive in national format even though you sent E.164, so it is not a reliable join key.

7.4 Verifying the signature

SwalaSMS signs the **exact JSON body** with HMAC SHA-256 and sends the digest in the `X-SwalaSMS-Signature` header. Verify before trusting anything in the payload.

```
$body = file_get_contents('php://input');
$expected = hash_hmac('sha256', $body, $yourWebhookSecret);
$actual = $_SERVER['HTTP_X_SWALASMS_SIGNATURE'] ?? '';
```



```
if (! hash_equals($expected, $actual)) {
    http_response_code(401);
    exit;
}

$payload = json_decode($body, true);
```

- Compute the HMAC over the **raw** request body, before any JSON parsing or re-serialisation. Re-encoding changes the bytes and breaks the signature.
- Use a constant-time comparison — `hash_equals` in PHP, `crypto.timingSafeEqual` in Node, `hmac.compare_digest` in Python.
- Reject with `401` on mismatch and do not process the payload.

7.5 Acknowledgement and retries

Your response	Treated as
Any <code>2xx</code>	Delivered. The attempt is complete.
Redirect (<code>3xx</code>)	Failed attempt.
Non- <code>2xx</code> (<code>4xx</code> , <code>5xx</code>)	Failed attempt.
Timeout	Failed attempt.

- Failed attempts are retried **up to four times** with increasing delay.
- Webhook delivery runs on a separate queue from SMS sending, so a slow endpoint cannot delay new messages.
- If an endpoint fails **10 times in a row**, SwalaSMS disables it and notifies the account by email and SMS. Re-enable it from the SMS Webhooks page.

Acknowledge first, work second. Verify the signature, write the event to a queue or table, return `2xx` immediately, and do the real processing asynchronously. A handler that does slow work inline will eventually time out, and ten consecutive timeouts disables the endpoint.

Handle duplicates. Because retries exist, the same event can arrive more than once. Make the handler idempotent — key on `message_id` plus `event` and ignore a repeat.

8. Error model

Every failure answers with JSON carrying `success: false`, the `request_uid` for the attempt, and a `message` written for a person. Quote the `request_uid` when contacting support.

8.1 Status codes

Status	Meaning	What to do
400	The body claims to be JSON but could not be parsed, so no fields were read and nothing was sent.	Check the payload for curly quotes, unterminated strings and missing commas.
401	Key missing, expired, revoked, calling from an address it is not allowed from, or without the permission the endpoint needs.	Check the key and its scopes in application settings.
404	The resource does not belong to this account.	Check the UID against your own records.
409	Idempotency key reused for different content, or a sender ID already requested for that country.	Read <code>error</code> . Do not retry blindly — neither case succeeds on repeat.
422	Understood but refused — validation, no credit, unapproved sender ID, unverified route, spam policy.	Read <code>message</code> ; it names the cause.
429	Too many requests.	Back off and retry; the limit is per key per minute.

8.2 Which failures are worth retrying

Status	Retry?	Why
429	Yes, with backoff	A transient limit. The same request will succeed once the window resets.
5xx / network	Yes, with backoff	Reuse the same <code>Idempotency-Key</code> so a duplicate cannot be created.
400	No	The payload is malformed. Fix it.
401	No	The key or its scopes are wrong. Fix configuration.
409	No	Refused deliberately. Read <code>error</code> and act on it.
422	No	A rule refused the request. Retrying changes nothing.

Retry safely. When retrying after a timeout, send the **same** `Idempotency-Key` and the **same** body. That is precisely the case the mechanism exists for: you get the original message back with `idempotent_replay: true` and nothing is sent twice.

8.3 Two kinds of 422

The status code is shared but the shape differs, and so does the fix.

	Validation 422	Rule 422
Trigger	A field is missing or malformed	The request is understood but refused by a platform rule
Shape	Carries an <code>errors</code> map keyed by field	Carries <code>message</code> , sometimes an <code>error</code> code
Examples	<code>recipient</code> absent; <code>sample_message</code> under 50 characters	Unapproved sender ID; no verified route; insufficient credit; spam policy; recipient opt-out
Fix	Correct the payload	Correct the account configuration, then retry

9. Playbooks

Ordered procedures for the situations an integration meets most often. Each one is built from the behaviour documented above; none of it requires undocumented knowledge.

9.1 First integration, from nothing to a live message

13. Create an application and issue a **sandbox** key. Store it in an environment variable.
14. Call `GET /api/v1/sms/profile` to confirm the key works and to see verified routes and balances.
15. Call `GET /api/v1/sender-ids/requirements`. Read `eligibility.can_request`, the `countries` list and `documents_required`.
16. If eligible, submit a sender ID with `POST /api/v1/sender-ids`. Poll `GET /api/v1/sender-ids` until `is_usable` is `true`.
17. Build the send path against the sandbox key: `POST /api/v1/sms/messages` with a UUID `Idempotency-Key`. Persist `data.uid`.
18. Configure a webhook endpoint against the sandbox key, verify the HMAC signature, and return `2xx` fast.
19. Exercise failure paths deliberately: omit a field to see a `422`, reuse an idempotency key with different content to see a `409`, break the JSON to see a `400`.
20. Top up SMS credits and confirm the balance with `GET /api/v1/balance`.
21. Issue a **live** key with the narrowest scopes the application needs. Swap the environment variable; no code should change.
22. Send one real message to a number you control. Confirm the webhook arrives and the status reads `delivered`.

9.2 Getting a sender ID approved

23. Call `GET /api/v1/sender-ids/requirements` first — always.
24. If `can_request` is `false`, purchase credits up to `minimum_purchase_amount` before doing anything else. Submitting anyway returns `sender_id_not_eligible`.
25. If the destination country is absent from `countries[]`, verify a phone number for it under SMS Routes. Submitting anyway returns `route_not_verified`.
26. Choose a name that passes the rules: 3 to 11 characters, letters and numbers with single spaces, not `SWALASMS` or `SWALA`. Pick the exact casing you want on the handset.
27. Write a real `sample_message` of 50 to 500 characters. Reviewers read it, so it should genuinely resemble the traffic you intend to send.
28. Choose one `purpose` from the list returned by the requirements call.
29. Attach documents only for requirements with `is_required: true`. On most platforms there are none.
30. Optionally set `protection_requested: true` and attach ownership proof to reserve the name.
31. Submit, then poll `GET /api/v1/sender-ids`. On `rejected`, read `rejection_reason` and resubmit a corrected request.

409 means stop. `sender_id_already_requested` means the name already exists for that country on this account. Retrying never succeeds — poll the list endpoint instead.

9.3 Running a bulk campaign

32. Normalise every recipient to E.164 and de-duplicate the list.
33. Split into batches of at most 1000 recipients.
34. Read `GET /api/v1/balance` and compare the total against your recipient count. Warn or stop if short.
35. Generate one `Idempotency-Key` per batch, derived from something stable in your own system so a retry reproduces it.
36. Set a meaningful `campaign_name` — it is what reports are grouped by.
37. POST to `/api/v1/sms/send-bulk`. Store the campaign `uid` and every `results[].message_uid`.
38. Read `failed_count` and `failures[]`. Per-recipient failures are normal and do not make the campaign unsuccessful.
39. Let webhooks resolve the outcomes. Reconcile anything still unresolved after a reasonable window with `GET /api/v1/sms/messages?channel=api`.

Scheduling. Set `scheduled_at` to a future ISO-8601 time to queue a campaign for later; omit it or send `null` to go now. Scheduled messages sit in `scheduled` status until their time arrives.

9.4 Reconciling delivery

Webhooks are the primary mechanism; polling is the safety net. A sound design uses both.

- On send, write a row with your own reference, the returned `uid`, the `request_uid` and `status: queued`.
- On webhook, verify the signature, match on `message_id`, and update the row to the final state. Ignore repeats.
- On a schedule — hourly is usually enough — query `GET /api/v1/sms/messages` filtered by `channel=api` and a date window, and resolve anything your database still shows as non-final.
- For a single stubborn message, read `GET /api/v1/sms/messages/{uid}`. It returns `failure_context` and the raw `delivery_report`, which the list view does not.

10. Troubleshooting

Symptom	Likely cause	Resolution
401 on every call	Key missing, revoked, or the placeholder was pasted unchanged.	Confirm the header reads <code>Authorization: Bearer swl_live_...</code> with a real key.
401 on one endpoint only	The key lacks that endpoint's scope.	Grant the scope in application settings, or use a key that holds it.
401 in production, fine in development	An address restriction on the key.	Add the production egress address to the key's allowed list.
400 right after copy-and-paste	Curly quotes replaced straight quotes.	Retype the quotes, or build the payload in code rather than pasting it.
422 naming the sender ID	The name is not approved on this account.	List approved names with <code>GET /api/v1/sender-ids</code> and use one where <code>is_usable</code> is <code>true</code> .
422 on a send that worked yesterday	The wallet ran out, or a route or sender ID changed state.	Check <code>GET /api/v1/sms/profile</code> and <code>GET /api/v1/balance</code> .
409 <code>idempotency_key_reused</code>	A key was reused for different content.	Generate a fresh UUID per message. The response names the earlier message.
429 under load	Per-key, per-minute limit exceeded.	Back off with jitter; move broadcasts to <code>send-bulk</code> .
404 on a status lookup	The UID is valid but belongs to another account.	Check the UID against your own records.
422 on a status lookup	The UID is not a UUID.	Fix the value being passed — usually a truncated or wrong field.
Webhooks stopped arriving	The endpoint failed 10 times in a row and was disabled.	Check email and SMS notifications, fix the handler, re-enable at SMS Webhooks.
Signature never matches	The body was parsed or re-serialised before hashing.	Hash the raw request body, and compare in constant time.
Everything reads <code>delivered</code> instantly	A sandbox key is in use.	Expected behaviour. Check <code>mode</code> in the send response; swap to a <code>swl_live_</code> key.
Messages send but none are billed	A sandbox key is in use.	Sandbox never consumes credits. Swap to a live key.

11. Quick reference

11.1 All endpoints

Method	Path	Scope	Purpose
GET	/balance	sms:read	Total and per-country SMS credit
POST	/sms/messages	sms:send	Send one SMS
GET	/sms/messages	sms:read	List messages with filters
POST	/sms/send-bulk	sms:send	Send one body to up to 1000 recipients
GET	/sms/messages/{uid}	sms:read	Status of one message
GET	/sender-ids/requirements	sender-ids:read	Eligibility, countries, documents, rules
GET	/sender-ids	sender-ids:read	List sender IDs and requests
POST	/sender-ids	sender-ids:write	Submit a sender ID for review
GET	/sms/profile	profile:read	Account, routes and balances

11.2 Alternate paths

Several endpoints answer at a second path with identical behaviour. Either form is correct; pick one and use it consistently.

Primary	Also answers at
/api/v1/balance	/api/v1/sms/balance
/api/v1/sms/messages (POST)	/api/v1/sms/quick-message
/api/v1/sender-ids/requirements	/api/v1/sms/sender-ids/requirements
/api/v1/sender-ids	/api/v1/sms/sender-ids

11.3 Limits

Item	Limit
Message body	1600 characters
Recipient field	32 characters
Sender ID field on send	32 characters
Sender ID name	3 to 11 characters

Item	Limit
Sample message	50 to 500 characters
Campaign name	120 characters
Bulk recipients per call	1000
List page size	50 by default, 100 maximum
Idempotency key memory	24 hours
Document formats	pdf, jpg, jpeg, png
Document size	Per <code>max_size_kb</code> in the requirements response (2048 KB in the published example)
Name-claim documents	3 maximum
Webhook retries	Up to 4 per event; endpoint disabled after 10 consecutive failures
Rate limit	Per key, per minute

12. Glossary

Term	Definition
Bearer token	The API key presented in the <code>Authorization</code> header on every request.
Channel	Where a message originated: <code>api</code> for integration traffic, <code>web</code> for messages composed in the portal.
Country wallet	A per-country balance of SMS units. The total balance is the sum across enabled wallets.
E.164	The international phone number format: leading plus, country code, subscriber number, no separators.
Idempotency key	A caller-supplied unique value that prevents a retry from sending the same message twice. Remembered for 24 hours.
<code>is_usable</code>	The flag on a sender ID that says it may be used for sending. Corresponds to <code>status: approved</code> .
Live key	A key prefixed <code>swl_live_</code> . Sends real SMS and consumes real credits.
Provider credit	Credit held with the upstream carrier route, distinct from the customer SMS credit in your wallet.
<code>request_uid</code>	An identifier for a single API attempt, present on every response. Quote it in support requests.
Sandbox key	A key prefixed <code>swl_test_</code> . Realistic responses, no real SMS, no credits consumed.
Sender ID	The name a recipient sees as the origin of the message. Registered per country and approved manually.
SMS route	A verified path to a destination country. A sender ID cannot be requested for a country without one.
Spam policy	The content check applied to every message body before it is accepted.
Unit	One billable SMS credit. A message reports the units it cost as <code>units</code> .
<code>uid</code>	The UUID of a message, campaign, sender ID or wallet. The message <code>uid</code> is the handle for status lookups.

Appendix A — Document control

Field	Value
Document title	SwalaSMS Developer API — Integration Knowledge Base
Revision	1.0
Compiled	14 September 2026
API version covered	v1.0.0
Source	https://swalasms.com/sms/developer/docs
Machine-readable spec	https://swalasms.com/sms/developer/openapi.json
API key management	https://swalasms.com/sms/api-keys
Webhook management	https://swalasms.com/sms/webhooks
Platform operator	SwalaSMS · developed by SwalaTech.com
Prepared for	FIXO — Home Services Made Simple

Maintenance notes

- The live reference is the authority. Re-check it before any significant integration change.
- The OpenAPI JSON is the better input for generating or regenerating client SDKs; this document is for people.
- Sections 9 to 11 contain implementation guidance derived from documented behaviour rather than verbatim platform text. Revisit them when the API version changes.
- No real API key, signing secret or account identifier appears anywhere in this document, and none should ever be added to it.